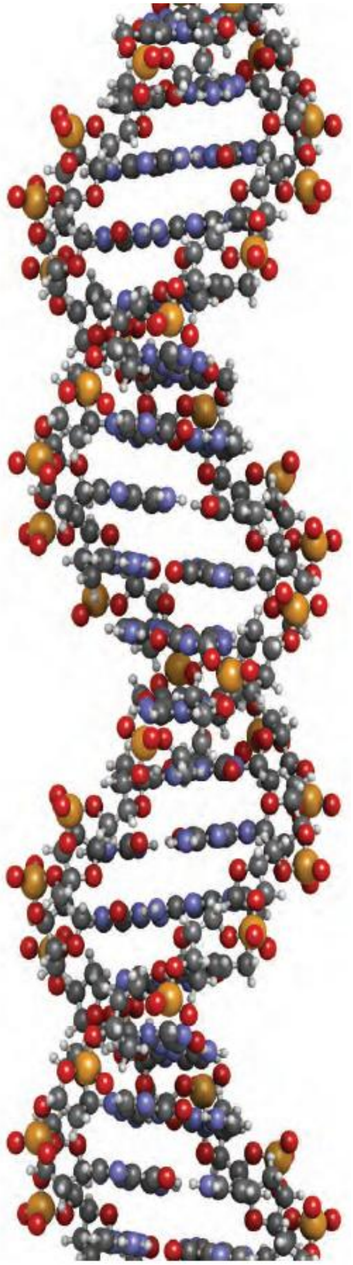
Lecture 4
SEQUENCE ALIGNMENT AND DYNAMIC PROGRAMMING



$O(n^2)$

1. The Needleman-Wunsch Algorithm

Úvod

- Biologický význam hľadania zarovnaní sekvencií **spočíva v schopnosti vyvodiť najpravdepodobnejší súbor evolučných udalostí**, ako sú bodové mutácie/nezhody a medzery (inzercie alebo delécie), ku ktorým došlo, aby sa jedna sekvencia transformovala na druhú. Aby sme tak urobili, najprv predpokladáme, že súbor transformácií s najnižšími nákladmi je najpravdepodobnejšou sekvenciou transformácií. Priradením nákladov každému typu transformácie (nezhoda alebo medzera), ktoré odrážajú ich príslušné úrovne evolučnej náročnosti, sa hľadanie optimálneho zarovnania redukuje na hľadanie súboru transformácií, ktoré vedú k najnižším celkovým nákladom.

- V predchádzajúcej prednáške sme ukázali, ako možno použiť dynamické programovanie na výpočet zarovnaní sekvencií v čase $O(n^2)$. Konkrétne sme definovali **Needleman-Wunschov algoritmus pre globálne zarovnanie**, navrhnutý Needlemanom a Wunschom, ktorý porovnáva kompletné sekvencie na úrovni nukleotidov.

- Problém hľadania zarovnania sekvencií možno pekne vyjadriť ako **algoritmus dynamického programovania,** pretože skóre zarovnania sú aditívne, čo znamená, že hľadanie zarovnania väčšej sekvencie možno nájsť rekurzívnym hľadaním zarovnaní menších podsekvencií. **Skóre sú uložené v matici, pričom jedna sekvencia zodpovedá stĺpcom a druhá sekvencia zodpovedá riadkom.** Každá bunka predstavuje transformáciu vyžadovanú medzi dvoma nukleotidmi zodpovedajúcimi riadku a stĺpcu danej bunky. Zarovnanie sa získa spätným sledovaním (traceback) cez maticu dynamického programovania.

$O(n^2)$

1. The Needleman-Wunsch Algorithm

Definícia 1. Pre dve dané sekvencie a a b s dĺžkami n_1 a n_2, zarovnanie $A(a,b)$ je postupnosť párov:

$$[(a_{i_1}, b_{j_1}); (a_{i_2}, b_{j_2}); \ldots; 1 \le i_1 \le i_2 \le \ldots n_1; 1 \le j_1 \le j_2 \le \ldots \le n_2]$$

Pre dané zarovnanie $A(a,b)$ definujeme:

$$M = |\{k : a_{i_k} = b_{j_k}\}|; \; X = |\{k : a_{i_k} \neq b_{j_k}\}|$$

a

$$S = |\{r : a_r \neq i_k\}| + |\{r : b_r \neq j_k\}|; k \in \mathbb{N}.$$

M sleduje počet zhodujúcich sa znakov, X počet nezhodujúcich sa znakov a S počet medzier (znakov nezarovnaných k inému znaku v druhej sekvencii).

Poznámka. Keďže každý znak musí byť zhodný, nezhodný alebo nezarovnaný, platí $2M + 2X + S = n_1 + n_2$.

Needleman-Wunschov algoritmus sa potom zameriava na rekurzívny výpočet matice S takej, že:

$$S(i,j) = \max\{S_{i-1,j} + s; S_{i,j-1} + s; S_{i-1,j-1} + xI(a_i = b_j)\}$$

kde I je indikačná funkcia identity (dávajúca 1, ak sú znaky rovnaké, a 0 v opačnom prípade).

Pre základné prípady platí pre všetky $i, j \in \mathbb{N}$: $S(i,0) = si$ a $S(0,j) = sj$.

1. The Needleman-Wunsch Algorithm

Zhrnutie: Needleman-Wunschov algoritmus pre globálne zarovnanie:

- Vypočítame skóre zodpovedajúce každej bunke v matici a zaznamenáme našu voľbu (memoizácia) v danom kroku, t. j. ktorá z horných, ľavých alebo diagonálnych buniek viedla k maximálnemu skóre pre aktuálnu bunku. Zostane nám matica plná optimálnych skóre na každej pozícii bunky spolu so smerovníkmi v každej bunke odrážajúcimi optimálnu voľbu, ktorá vedie k danej konkrétnej bunke.
- Optimálne zarovnanie potom môžeme získať spätným sledovaním z bunky v pravom dolnom rohu (ktorá obsahuje skóre zarovnania jednej kompletnej sekvencie s druhou) sledovaním smerovníkov odrážajúcich lokálne optimálne voľby a následnou konštrukciou zarovnania zodpovedajúceho optimálnej ceste sledovanej v matici.
- Čas behu Needleman-Wunschovho algoritmu je $O(n^2)$, pretože pre každú bunku v matici vykonávame konečné množstvo výpočtov. Vypočítame 3 hodnoty pomocou už vypočítaných skóre a potom vezmeme maximum z týchto hodnôt, aby sme našli skóre zodpovedajúce danej bunke, čo je operácia s konštantným časom .
- Na zaručenie správnosti je potrebné vypočítať náklady pre každú bunku matice. Je možné, že optimálne zarovnanie môže byť na začiatku tvorené zlým zarovnaním (pozostávajúcim z medzier a nezhôd), po ktorom nasleduje mnoho zhôd, čo z neho robí celkovo najlepšie zarovnanie. Toto sú prípady, ktoré prechádzajú cez hranice našej matice zarovnania. Teda na zaručenie optimálneho globálneho zarovnania musíme vypočítať každý záznam matice.

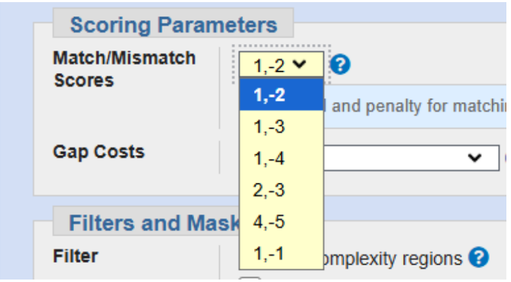
Needleman-Wunschov algoritmus sa zvyčajne aplikuje, keď je známe, že sekvencie sú homológne (t. j. sekvencie pochádzajú z organizmov, ktoré zdieľajú spoločného predka). **Prístup dynamického programovania je uprednostňovaný pred chamtivým (greedy) algoritmom**, ktorý jednoducho vyberá prechod s minimálnymi nákladmi v každom kroku, pretože chamtivý algoritmus nezaručuje, že celkový výsledok poskytne optimálne zarovnanie alebo zarovnanie s najnižšími nákladmi.

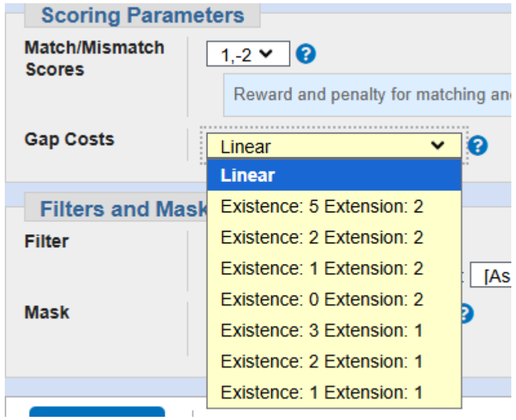
Needleman-Wunschov algoritmus je menej užitočný na porovnávanie sekvencií s prestavbami alebo inverziami, alebo na zarovnávanie novo osekvenovaného génu voči referenčným génom v známom genóme, čo je známe ako vyhľadávanie v databáze. V praxi môžeme tiež často obmedziť priestor zarovnania, ktorý sa má preskúmať, ak vieme, že niektoré zarovnania sú jasne suboptimálne.

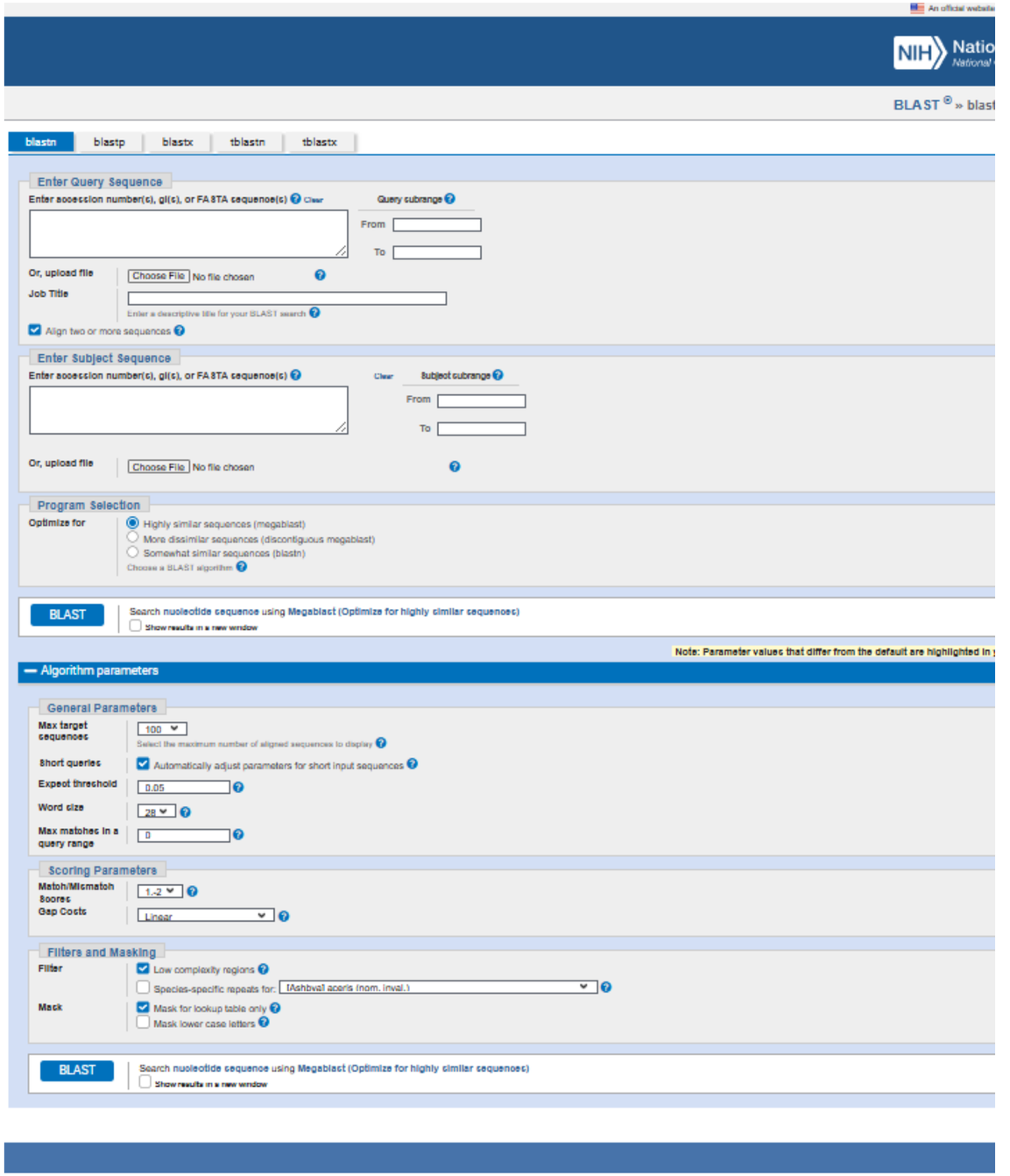
1. The Needleman-Wunsch Algorithm

BLAST – nastavenie parametrov v prvom projekte

- https://blast.ncbi.nlm.nih.gov/Blast.cgi







Optimalizácie

Problém škálovateľnosti

- Štandardný dynamický algoritmus s ***O(mn),*** ktorý sme prezentovali na minulej prednáške, je oveľa rýchlejší než stratégia hrubej sily založená na vymenúvaní všetkých možných zarovnaní a **funguje dobre pre sekvencie dlhé až 10 kilobáz.** je efektívny do 10 kb, ale pri celých genómoch (milióny báz) prestáva byť realizovateľný. Aby sme mohli zarovnávať oveľa väčšie sekvencie, môžeme vykonať nejaké modifikácie algoritmu a ďalej zlepšiť jeho výkon.
- **Cieľ:** Znížiť počet výpočtov v matici podproblémov ***F***.

Pásmové DP (Banded Dynamic Programming)

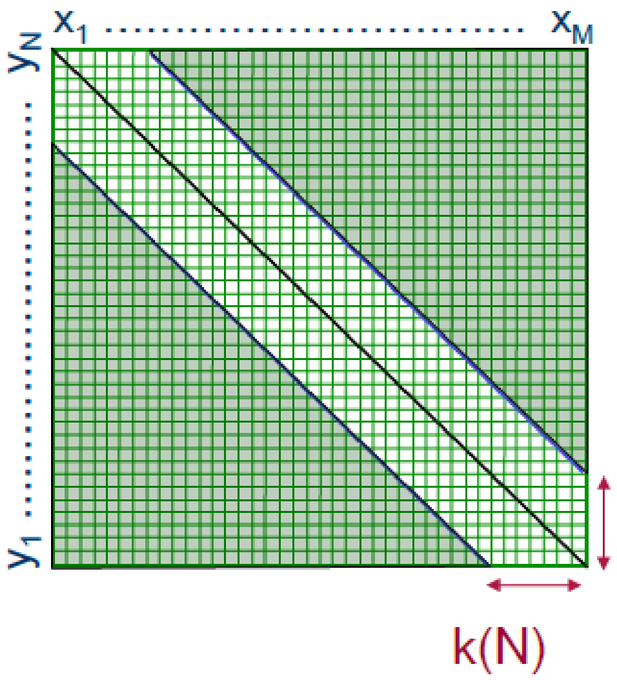
- **Kľúčová idea:** Ignorujeme „mierne nudné zarovnania“ (MBAs), ktoré obsahujú príliš veľa medzier.
- **Algoritmus:** Výpočet sa obmedzuje len na polia v určitej vzdialenosti od hlavnej diagonály.
- **Zložitosť**: Časové a priestorové náklady klesajú na ***O(m+n)W***.

Dôležité charakteristiky

- **Heuristika:** Na rozdiel od plného DP už nezaručuje nájdenie absolútneho optima.
- **Využitie:** Dosahuje dolnú hranicu (lower bound) skóre, čo umožňuje neskoršie orezávanie nevýhodných ciest v matici.

Dynamické programovanie

- Jednou z možných optimalizácií je ignorovať „mierne nudné zarovnania“ (MBAs – Mildly Boring Alignments), alebo zarovnania, ktoré majú príliš veľa medzier.
- **Explicitne povedané, môžeme sa obmedziť na zotrvanie v rámci určitej vzdialenosti W od diagonály v matici podproblémov F.** To znamená, že predpokladáme, že optimálna cesta v F z bodu $F_{0,0}$ do $F_{m,n}$ sa nachádza v rámci vzdialenosti W pozdĺž diagonály. To znamená, že rekurziu je potrebné aplikovať iba na polia v F, ktoré sú vo vzdialenosti W okolo diagonály, čo prináša časové a priestorové náklady vo výške $O((m+n)W)$.
- **Všimnite si však, že táto stratégia je heuristická a už viac nezaručuje nájdenie optimálneho zarovnania.** Namiesto toho dosahuje dolnú hranicu (lower bound) optimálneho skóre. To môžeme využiť v následnom kroku, kde zahodíme rekurzie v matici F, ktoré vzhľadom na túto dolnú hranicu nemôžu viesť k optimálnemu zarovnaniu.



Poznámka: Čo presne znamená termín „Mildly Boring Alignments“ (MBA)?
Je to neformálny termín pre zarovnania, ktoré sa príliš vzďaľujú od hlavnej diagonály. Biologicky to znamená, že jedna sekvencia by musela mať obrovskú medzeru oproti druhej. Ak predpokladáme, že evolučne sú sekvencie príbuzné bez masívnych štrukturálnych zmien, tieto "vzdialené" cesty sú pre nás nezaujímavé ("nudné"), pretože ich skóre bude aj tak veľmi nízke.

1. The Needleman-Wunsch Algorithm

Je však možné nájsť optimálne zarovnanie aj s použitím iba lineárneho priestoru!
Cieľom je použiť metódu „rozdeľ a panuj“ (divide and conquer), aby sme v každom kroku vypočítali štruktúru optimálneho zarovnania pre jeden prvok matice.
Kľúčovou myšlienkou je, že zarovnávanie pomocou dynamického programovania môže postupovať rovnako ľahko aj v opačnom smere, začínajúc v pravom dolnom rohu a končiac vľavo hore. Takže ak sa matica rozdelí na polovicu, dopredný priechod aj spätný priechod môžu bežať súčasne a stretnúť sa v strednom stĺpci. V bode kríženia môžeme sčítať tieto dve skóre zarovnaní; bunka v strednom stĺpci s maximálnym skóre musí ležať na celkovej optimálnej ceste.

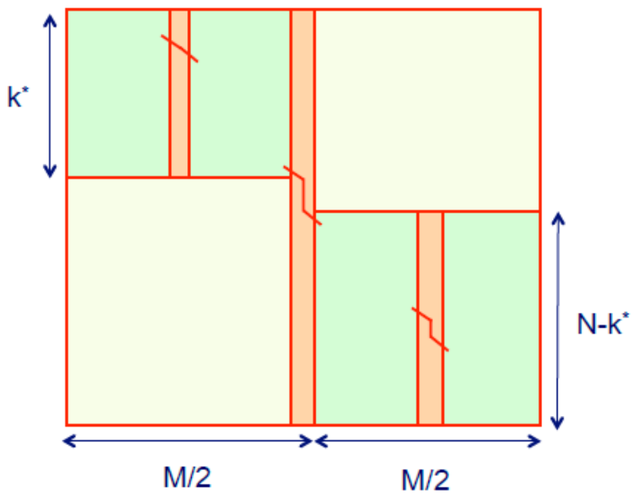
Optimalizácia pamäte: Lineárny priestor (Hirschberg)

Obmedzenia štandardného Tracebacku

- Výpočet samotného **skóre** vyžaduje len $O(m)$ priestoru (stačí nám predchádzajúci stĺpec).
- Avšak rekonštrukcia cesty vyžaduje uloženie smerovníkov pre celú maticu, čo stojí $O(mn)$ priestoru.

Princíp obojsmerného výpočtu

- DP algoritmus funguje identicky v **doprednom smere** (od $F_{0,0}$) aj v **spätnom smere** (od $F_{m,n}$).
- Ak rozdelíme maticu v strednom stĺpci ($n/2$), dopredný a spätný priechod sa tam stretnú.
- **Bod kríženia:** Bunka v strednom stĺpci s maximálnym súčtom dopredného a spätného skóre je garantovanou súčasťou optimálnej cesty.



1. The Needleman-Wunsch Algorithm

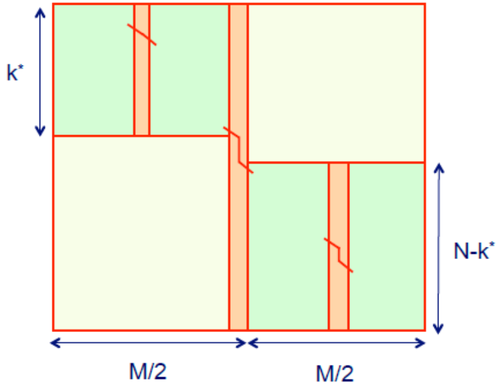
Algoritmus „Rozdeľ a panuj“ (Divide and Conquer)

Proces rekurzívneho delenia

1. Nájdeme index riadku u v strednom stĺpci, ktorý leží na optimálnej ceste (čas $O(mn)$, priestor $O(m)$).
2. Maticu rozdelíme na dve menšie podmatice:
   - **Horná ľavá:** rozmer $u \times (n/2)$.
   - **Dolná pravá:** rozmer $(m-u) \times (n/2)$.
3. Postup rekurzívne opakujeme na čoraz menšie časti, kým nezískame celú optimálnu cestu.

Komplexita a efektivita

- **Časová zložitosť:** $O(mn) + O(mn/2) + O(mn/4) + \ldots$ konverguje k $O(2mn) = O(mn)$.
- **Priestorová zložitosť:** Znižuje sa na $O(m+n)$.
- **Výsledok:** Získame optimálne zarovnanie v lineárnom priestore bez asymptotického spomalenia výpočtu (len s konštantným faktorom 2).



Tento proces môžeme opísať formálnejšie a kvantitatívnejšie.

- Najprv vypočítajte index riadku $u \in \{1, \ldots, m\}$, ktorý sa nachádza na optimálnej ceste pri prechode $n/2$-tým stĺpcom.
- Pre $1 \le i \le m$ a $n/2 \le j \le n$ nech $C_{i,j}$ označuje index riadku, ktorý sa nachádza na optimálnej ceste k $F_{i,j}$ pri prechode $n/2$-tým stĺpcom.
- Potom, zatiaľ čo aktualizujeme stĺpce F zľava doprava, môžeme tiež aktualizovať stĺpce C zľava doprava. Takto sme v čase $O(mn)$ a priestore $O(m)$ schopní vypočítať skóre $F_{m,n}$ a tiež $C_{m,n}$, ktoré sa rovná indexu riadku $u \in \{1, \ldots, m\}$, ktorý sa nachádza na optimálnej ceste pri prechode $n/2$-tým stĺpcom.
- **Teraz prichádza na rad myšlienka „rozdeľ a panuj“.** Zopakujeme vyššie uvedený postup pre hornú ľavú podmaticu $u \times n/2$ matice F a taktiež zopakujeme vyššie uvedený postup pre dolnú pravú podmaticu $(m-u) \times n/2$ matice F. To možno vykonať s použitím alokovaného lineárneho priestoru $O(m+n)$.
- Čas behu pre hornú ľavú podmaticu je $O(u \cdot n/2)$ a čas behu pre dolnú pravú podmaticu je $O((m-u) \cdot n/2)$, čo po sčítaní dáva čas behu $O(mn/2) = O(mn)$.
- Pokračujeme v opakovaní vyššie uvedeného postupu pre čoraz menšie podmatice matice F, zatiaľ čo zbierame viac a viac prvkov zarovnania s optimálnym skóre. Celkový čas behu je $O(mn) + O(mn/2) + O(mn/4) + \ldots = O(2mn) = O(mn)$. Takže bez obetovania celkového času behu (až na konštantný faktor) vedie metóda „rozdeľ a panuj“ k riešeniu v lineárnom priestore.

1. The Needleman-Wunsch Algorithm

Viacnásobné zarovnanie (Multiple Alignment)

Zarovnávanie troch sekvencií

Teraz, keď sme videli, ako zarovnať pár sekvencií, je prirodzené rozšíriť túto myšlienku na viacero sekvencií. Predpokladajme, že by sme chceli nájsť optimálne zarovnanie 3 sekvencií. Ako by sme mohli postupovať?

Pripomeňme si, že keď zarovnávame dve sekvencie S a T, vyberáme maximum z troch možností pre finálnu pozíciu zarovnania (sekvencia T zarovnaná voči medzere, sekvencia S zarovnaná voči medzere, alebo sekvencia S zarovnaná voči sekvencii T):

$$F_{i,j} = \max \begin{cases} F_{i,j-1} + d \\ F_{i-1,j} + d \\ F_{i-1,j-1} + s(S_i, T_j) \end{cases}$$

Pre tri sekvencie S, T a U existuje sedem možností pre finálnu pozíciu zarovnania. To znamená, že existujú tri spôsoby, ako mať dve medzery na finálnej pozícii, tri spôsoby, ako mať jednu medzeru, a jeden spôsob, ako mať zarovnané všetky tri sekvencie $\binom{3}{1} + \binom{3}{2} + \binom{3}{3} = 7$.

Pravidlo aktualizácie je teraz:

$$F_{i,j,k} = \max \begin{cases} F_{i-1,j,k} + s(S_i, -, -) \\ F_{i,j-1,k} + s(-, T_j, -) \\ F_{i,j,k-1} + s(-, -, U_k) \\ F_{i-1,j-1,k} + s(S_i, T_j, -) \\ F_{i-1,j,k-1} + s(S_i, -, U_k) \\ F_{i,j-1,k-1} + s(-, T_j, U_k) \\ F_{i-1,j-1,k-1} + s(S_i, T_j, U_k) \end{cases}$$

Pravidlo aktualizácie je teraz:

$$F_{i,j,k} = \max \begin{cases} F_{i-1,j,k} + s(S_i, -, -) \\ F_{i,j-1,k} + s(-, T_j, -) \\ F_{i,j,k-1} + s(-, -, U_k) \\ F_{i-1,j-1,k} + s(S_i, T_j, -) \\ F_{i-1,j,k-1} + s(S_i, -, U_k) \\ F_{i,j-1,k-1} + s(-, T_j, U_k) \\ F_{i-1,j-1,k-1} + s(S_i, T_j, U_k) \end{cases}$$

kde s je funkcia popisujúca skóre pre medzeru, zhodu a nezhodu.

Tento prístup je však exponenciálny vzhľadom na počet sekvencií, ktoré zarovnávame. Ak máme k sekvencií dĺžky n, výpočet optimálneho zarovnania pomocou k-rozmernej matice dynamického programovania trvá $O((2n)^k)$ (faktor 2 vyplýva zo skutočnosti, že k-rozmerná kocka má 2^k vrcholov, takže pre každý záznam v matici skóre musíme vziať maximum z $2^k - 1$ susedných buniek). Ako si viete predstaviť, tento algoritmus sa s pribúdajúcim počtom sekvencií rýchlo stáva nepraktickým.

Matematická zložitosť

- Pre k sekvencií potrebujeme **k-rozmernú maticu**.
- Počet susedných buniek, z ktorých hľadáme maximum, je $2^k - 1$.
- **Časová zložitosť:** $O((2n)^k)$ – exponenciálny nárast.
- **Záver:** Pre viac ako 3-4 sekvencie je výpočet čistým dynamickým programovaním v praxi nerealizovateľný.

1. The Needleman-Wunsch Algorithm

Heuristika: Progresívne zarovnanie

Jeden bežne používaný prístup pre viacnásobné zarovnanie sekvencií sa nazýva **progresívne viacnásobné zarovnanie.** Predpokladajme, že poznáme evolučný strom spájajúci každú z našich sekvencií. Potom začneme vykonaním párového zarovnania dvoch najviac príbuzných sekvencií. Toto počiatočné zarovnanie sa nazýva „seed“ (zárodočné) zarovnanie. Potom pokračujeme zarovnaním ďalšej najbližšej sekvencie k tomuto zárodku a toto nové zarovnanie nahradí ten pôvodný zárodok. Tento proces pokračuje, až kým sa nevytvorí finálne zarovnanie.
V praxi **vo všeobecnosti evolučný strom (alebo vodiaci strom – guide tree) nepoznáme;** táto technika sa zvyčajne spája s nejakým typom klastrovacieho algoritmu, ktorý môže použiť mieru podobnosti s nízkym rozlíšením na generovanie odhadu stromu.
Hoci je čas behu tohto heuristického prístupu oveľa lepší v porovnaní s predchádzajúcou metódou (polynomiálny vzhľadom na počet sekvencií namiesto exponenciálneho), už nemôžeme zaručiť, že finálne zarovnanie je optimálne.

Princíp "Rozdeľ a panuj" v evolúcii

- Namiesto hľadania globálneho optima pre všetky sekvencie naraz, postupujeme po krokoch.
- **Vodiaci strom (Guide Tree):** Určuje poradie, v akom budeme sekvencie spájať (zvyčajne od najpodobnejších k najvzdialenejším).

Algoritmus krok za krokom

1. **Párové zarovnanie:** Vyberú sa dve najbližšie sekvencie a vytvorí sa základné (**seed**) zarovnanie.
2. **Rozširovanie:** K existujúcemu zárodku sa pridá ďalšia najbližšia sekvencia.
3. **Iterácia:** Proces sa opakuje, kým nie sú zahrnuté všetky sekvencie.

Výhody a limity

- **Výhoda:** Polynomiálna časová zložitosť (výrazne rýchlejšie).
- **Nevýhoda:** Už nie je garantované globálne optimum (chyba v skorom kroku sa prenáša ďalej).

Zarovnávanie sekvencie voči profilu (Profile Alignment)

Ako pridať sekvenciu k existujúcemu bloku?

Existujú dve hlavné stratégie, ako "napojiť" novú sekvenciu na už hotový seed:

1. **Metóda Feng-Doolittle:**
   - Nová sekvencia sa porovná so všetkými sekvenciami v zárodku samostatne.
   - Pridá sa na základe najlepšieho nájdeného párového skóre.
   - Už existujúce medzery v bloku zostávajú zachované.
2. **Súčet párov (Sum of Pairs - SP):**
   - Navrhne sa špeciálna skórovacia funkcia pre celé stĺpce.
   - Skóre stĺpca je definované ako súčet skóre všetkých možných dvojíc v danom stĺpci.

Súčasný výskum

- MSA je stále "horúcou" témou. Moderné nástroje (ako ClustalW, MAFFT či T-Coffee) kombinujú tieto heuristiky s pokročilými klastrovacími algoritmami.

1. The Needleman-Wunsch Algorithm

Vizualizácia: 3D matica (Kocka)

Predstavte si kocku zloženú z malých buniek. Každá os (x, y, z) predstavuje jednu z troch sekvencií. $F_{i,j,k}$, musíme sa teda pozrieť na všetkých jej susedov, z ktorých sme do nej mohli prísť.

- **V 2D (plocha):** Máme 3 susedov (horný, ľavý, diagonálny).
- **V 3D (objem):** Máme **7 susedov**. Sú to:
  - 3 susedia cez steny (posun v jednej sekvencii – 2 medzery).
  - 3 susedia cez hrany (posun v dvoch sekvenciách – 1 medzera).
  - 1 sused cez vnútornú diagonálu (posun vo všetkých troch – match/mismatch).

Prečo zložitosť rastie exponenciálne?

Ak by sme mali k sekvencií, počet susedov, ktorých musíme v každom kroku skontrolovať, rastie podľa vzorca $2^k - 1$.

Počet sekvencií (k)	Počet susedov (smerov)	Poznámka
2	$2^2 - 1 = \mathbf{3}$	Klasický Needleman-Wunsch (zvládnuteľné).
3	$2^3 - 1 = 7$	3D kocka (stále realizovateľné pre krátke sekvencie).
10	$2^{10} - 1 = \mathbf{1023}$	Extrémne náročné na každú jednu bunku matice.
20	$2^{20} - 1 \approx \mathbf{1000000}$	Prakticky nemožné vypočítať jediný krok.

Okrem počtu smerov rastie aj **počet buniek** v matici. Pre 10 sekvencií dĺžky 100 báz by matica mala 100^{10} buniek, čo je číslo presahujúce kapacitu akéhokoľvek dnešného úložiska.

1. The Needleman-Wunsch Algorithm

Pseudokód pre Needleman-Wunschov algoritmus

Prvá úloha v prvej sade problémov od vás vyžaduje dokončenie implementácie Needleman-Wunschovho (NW) algoritmu, pričom funkčný kód je zámerne vynechaný.

Problém: Pre dané dve sekvencie S a T dĺžok m a n, substitučnú maticu vU s hodnotami skóre pre zhody a penalizáciu za medzeru G, určte optimálne zarovnanie S a T a skóre tohto zarovnania.

Algoritmus:

1. **Vytvorte dve matice** A **a** B s rozmermi $(m+1)\times(n+1)$. A bude matica skóre a B bude matica spätného sledovania (traceback matrix). Prvok (i,j) matice A bude uchovávať skóre optimálneho zarovnania sekvencií $S[1;\ldots;i]$ a $T[1;\ldots;j]$ a prvok (i,j) matice B bude uchovávať smerovník na prvok, z ktorého bolo toto optimálne zarovnanie vytvorené.

2. **Inicializujte prvý riadok a stĺpec matice skóre** A tak, aby skóre zohľadňovalo penalizácie za medzery, a inicializujte prvý riadok a stĺpec matice spätného sledovania B zrejmým spôsobom.

3. **Prechádzajte prvkami** (i,j) **matice** A v nejakom rozumnom poradí a určujte optimálne zarovnanie sekvencií $S[1;\ldots;i]$ a $T[1;\ldots;j]$ s využitím prvkov $(i-1;j-1)$, $(i-1;j)$ a $(i;j-1)$. Nastavte smerovník v matici B na zodpovedajúci prvok, z ktorého bolo optimálne zarovnanie v bode (i,j) vytvorené.

4. **Keď sú všetky prvky matíc** A **a** B **dokončené**, skóre optimálneho zarovnania možno nájsť v prvku (m,n) matice A.

5. **Konštruujte optimálne zarovnanie** sledovaním cesty smerovníkov začínajúc v prvku (m,n) matice B a končiac v prvku $(0,0)$ matice B.

1. The Needleman-Wunsch Algorithm

Pseudokód pre Needleman-Wunschov algoritmus

Prvá úloha v prvej sade problémov od vás vyžaduje dokončenie implementácie Needleman-Wunschovho (NW) algoritmu, pričom funkčný kód je zámerne vynechaný.

Problém: Pre dané dve sekvencie S a T dĺžok m a n, substitučnú maticu vU s hodnotami skóre pre zhody a penalizáciu za medzeru G, určte optimálne zarovnanie S a T a skóre tohto zarovnania.

Algoritmus:

1. **Vytvorte dve matice** A **a** B s rozmermi $(m+1)\times(n+1)$. A bude matica skóre a B bude matica spätného sledovania (traceback matrix). Prvok (i,j) matice A bude uchovávať skóre optimálneho zarovnania sekvencií $S[1;\ldots;i]$ a $T[1;\ldots;j]$ a prvok (i,j) matice B bude uchovávať smerovník na prvok, z ktorého bolo toto optimálne zarovnanie vytvorené.

2. **Inicializujte prvý riadok a stĺpec matice skóre** A tak, aby skóre zohľadňovalo penalizácie za medzery, a inicializujte prvý riadok a stĺpec matice spätného sledovania B zrejmým spôsobom.

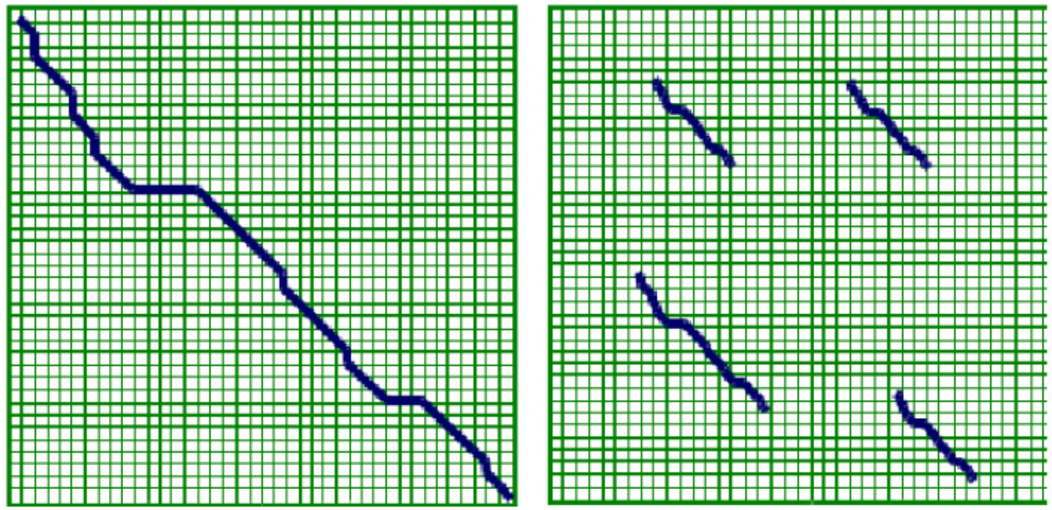
3. **Prechádzajte prvkami** (i,j) **matice** A v nejakom rozumnom poradí a určujte optimálne zarovnanie sekvencií $S[1;\ldots;i]$ a $T[1;\ldots;j]$ s využitím prvkov $(i-1;j-1)$, $(i-1;j)$ a $(i;j-1)$. Nastavte smerovník v matici B na zodpovedajúci prvok, z ktorého bolo optimálne zarovnanie v bode (i,j) vytvorené.

4. **Keď sú všetky prvky matíc** A **a** B **dokončené**, skóre optimálneho zarovnania možno nájsť v prvku (m,n) matice A.

5. **Konštruujte optimálne zarovnanie** sledovaním cesty smerovníkov začínajúc v prvku (m,n) matice B a končiac v prvku $(0,0)$ matice B.

Porovnanie globálneho, lokálneho a semi-globálneho zarovnania

- Globálne zarovnanie je definované ako zarovnanie dvoch reťazcov s a t od začiatku až do konca (Obrázok a). Lokálne zarovnanie reťazcov s a t je zarovnanie podreťazcov s s podreťazcami t (Obrázok b).
- Často nás viac zaujíma hľadanie **lokálnych zarovnaní, pretože zvyčajne nepoznáme hranice génov a pretože dlhé segmenty často podliehajú prestavbám.** Okrem toho môže byť niekedy konzervovaná len malá doména génu. V takýchto prípadoch nechceme vynucovať, aby sa zarovnali aj ostatné (potenciálne nehomológne) časti sekvencie.



(a) Example of a global alignment solution (b) Example of a local alignment solution

Needleman-Wunsch algorithm

$$
\begin{aligned}
Initialization &: \quad F(0,0) = 0 \\
Iteration &: \quad F(i,j) = \max \begin{cases} F(i-1,j) - d \\ F(i,j-1) - d \\ F(i-1,j-1) + s(x_i, y_j) \end{cases} \\
Termination &: \quad Bottom\ right
\end{aligned}
$$

- Pre nájdenie lokálnych zarovnaní stačí Needleman-Wunschov algoritmus len mierne upraviť tak, aby **začal odznova a našiel nové lokálne zarovnanie vždy, keď skóre existujúceho zarovnania klesne do záporných hodnôt.**
- Keďže lokálne zarovnanie môže začať kdekoľvek, inicializujeme prvý riadok a stĺpec v matici nulami. Krok iterácie je upravený tak, aby zahŕňal nulu, čím sa zahrnie možnosť, že začať nové zarovnanie bude výhodnejšie (lacnejšie), než mať veľa nezhôd.
- Okrem toho, keďže zarovnanie môže skončiť kdekoľvek, musíme prejsť celú maticu, aby sme našli optimálne skóre zarovnania (nielen v pravom dolnom rohu). Zvyšok algoritmu, vrátane spätného sledovania (traceback), zostáva nezmenený, pričom **traceback indikuje koniec pri nule, čo označuje začiatok optimálneho zarovnania.**

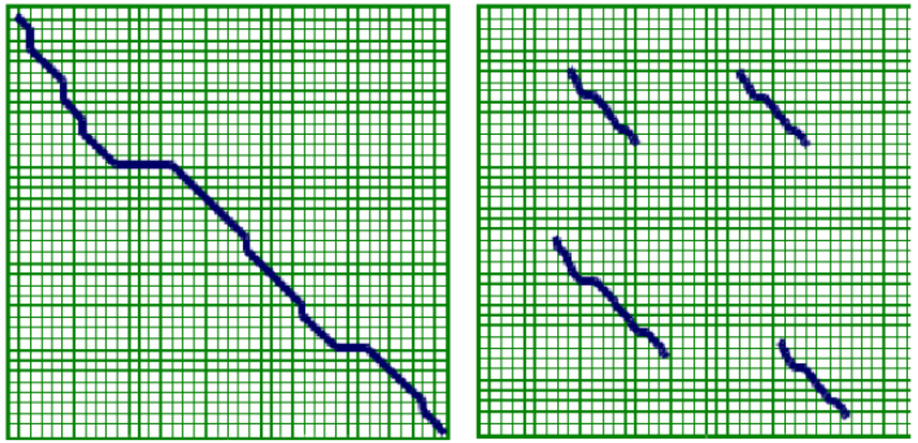
Smith-Waterman algorithm

$$
\begin{aligned}
Initialization &: \quad F(i,0) = 0 \\
& \quad\quad\ \ F(0,j) = 0 \\
Iteration &: \quad F(i,j) = \max \begin{cases} 0 \\ F(i-1,j) - d \\ F(i,j-1) - d \\ F(i-1,j-1) + s(x_i, y_j) \end{cases} \\
Termination &: \quad Anywhere
\end{aligned}
$$

Smith-Watermanov algoritmus pre lokálne zarovnanie

Kľúčové modifikácie NW algoritmu:

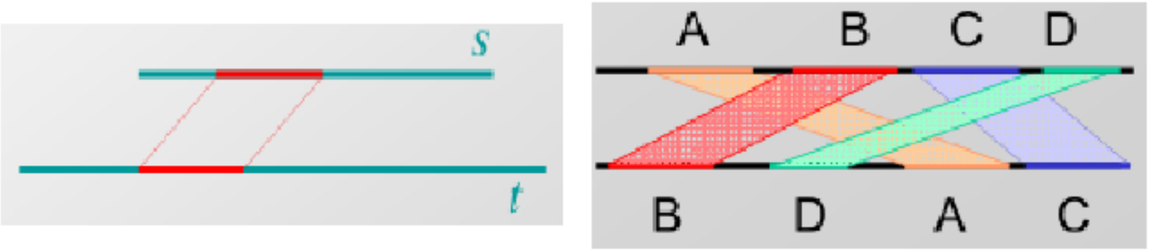
1. **Inicializácia:** Prvý riadok a stĺpec sú vyplnené **nulami** (zarovnanie môže začať kdekoľvek).
2. **Skórovanie:** Do rekurzívneho vzťahu pribúda **0** ako štvrtá možnosť.
   - $F_{i,j} = \max(0, F_{i-1,j-1} + s, F_{i-1,j} + d, F_{i,j-1} + d)$
   - *Logika:* Ak by malo byť skóre záporné, radšej začneme nové zarovnanie od nuly.
3. **Ukončenie:** Optimálne skóre hľadáme v **celej matici**, nielen v pravom dolnom rohu.
4. **Traceback:** Začína v bunke s maximálnym skóre a končí v prvej bunke s hodnotou 0.



(a) Example of a global alignment solution (b) Example of a local alignment solution

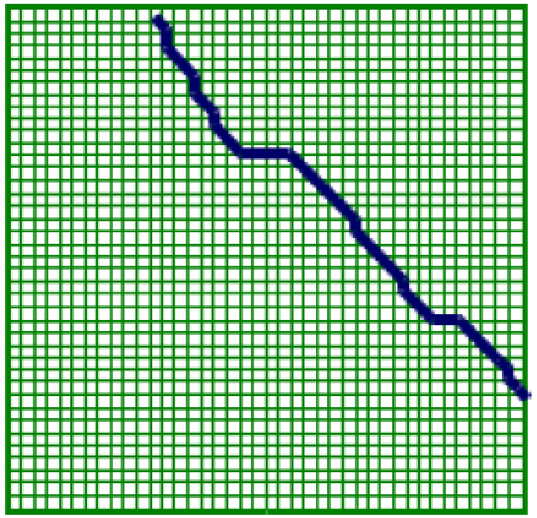
Porovnanie globálneho, lokálneho a semi-globálneho zarovnania

- Lokálne zarovnanie je užitočné aj pri hľadaní malého génu vo veľkom chromozóme.



Using local alignments to detect rearrangements

- Semi-globálne zarovnanie je v podstate globálne zarovnanie, v ktorom počiatočné koncové medzery nie sú penalizované



Example of a semi-global alignment solution

Needleman-Wunsch algorithm

$$
\begin{aligned}
Initialization &: \quad F(0,0) = 0 \\
Iteration &: \quad F(i,j) = \max \begin{cases} F(i-1,j) - d \\ F(i,j-1) - d \\ F(i-1,j-1) + s(x_i, y_j) \end{cases} \\
Termination &: \quad Bottom\ right
\end{aligned}
$$

- Pre nájdenie **semi-globálneho zarovnania** je kľúčovou myšlienkou, že si neželáme penalizovať počiatočné a koncové medzery. Preto **inicializujeme horný riadok a najľavejší stĺpec na nulu**, aby sme naznačili rovnaké náklady bez ohľadu na počiatočný bod, a končíme buď v spodnom riadku, alebo v najpravejšom stĺpci, aby sme zabránili penalizácii koncových medzier.

Semi-globálneho zarovnania

$$
\begin{aligned}
Initialization &: \quad F(i,0) = 0 \\
& \quad\ \ F(0,j) = 0 \\
Iteration &: \quad F(i,j) = \max \begin{cases} F(i-1,j) - d \\ F(i,j-1) - d \\ F(i-1,j-1) + s(x_i, y_j) \end{cases} \\
Termination &: \quad Bottom\ row\ or\ Right\ column
\end{aligned}
$$

2. Úvod do rýchleho zarovnávania a vyhľadávania

Algoritmické variácie na úsporu času a pamäte

1. Banded DP (Ohraničený priestor)

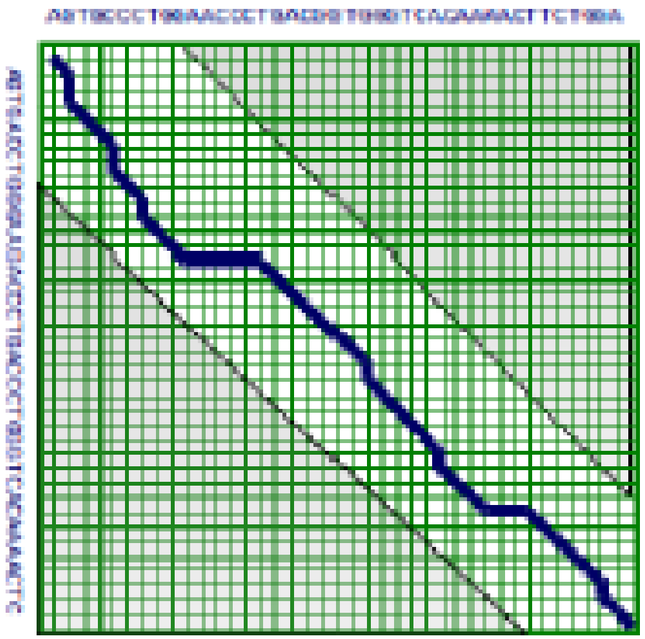
- **Princíp:** Skúmame len okolie diagonály (rádius k).
- **Výhoda:** Rýchlosť $O(k \cdot m)$ a nízka pamäť.
- **Nevýhoda: Heuristika.** Môže minúť optimálnu cestu, ak sa táto nachádza ďaleko od diagonály.

2. Lineárna pamäť (Hirschbergov algoritmus)

- **Skóre:** Vyžaduje len dva stĺpce pamäte $O(m)$.
- **Celé zarovnanie:** Využíva **Rozdeľ a panuj**.
  - Nájde stredný bod optimálnej cesty sčítaním dopredného a spätného skóre.
  - Rekurzívne delí maticu na menšie časti.
- **Zložitosť:** Pamäť $O(m)$, Čas $O(2mn)$. Je to **presný algoritmus** (nie heuristika), hoci je 2x pomalší.

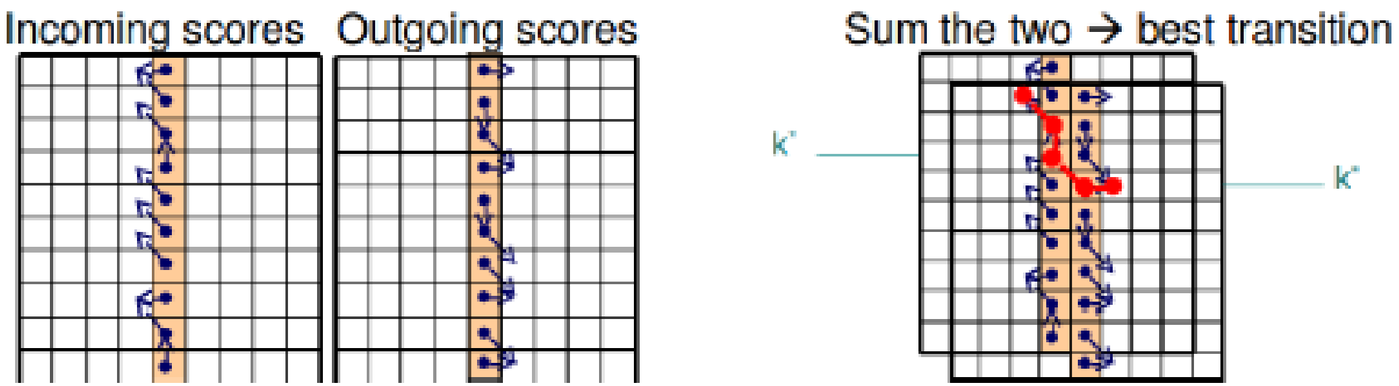
Algoritmické variácie

- Jednou z metód na úsporu času je **myšlienka ohraničenia priestoru** zarovnaní, ktorý sa má preskúmať. Ide o to, že dobré zarovnania vo všeobecnosti zostávajú blízko diagonály matice. Môžeme teda preskúmať iba bunky matice v okruhu od diagonály. Problémom tejto modifikácie je, že ide o heuristiku a môže viesť k suboptimálnemu riešeniu, pretože nezahŕňa hraničné prípady spomenuté na začiatku kapitoly. Napriek tomu to v praxi funguje veľmi dobre. Navyše, v závislosti od vlastností skórovacej matice môže byť možné argumentovať správnosťou algoritmu s ohraničeným priestorom.



2. Úvod do rýchleho zarovnávania a vyhľadávania

Ak použijeme princíp „rozdeľ a panuj“, môžeme skutočne nájsť optimálne zarovnanie v lineárnom priestore. Myšlienka spočíva v tom, že vypočítame optimálne zarovnania z oboch strán matice, t. j. zľava doprava a naopak. Nech $u = \lfloor n/2 \rfloor$. Povedzme, že dokážeme identifikovať v také, že bunka (u, v) sa nachádza na optimálnej ceste zarovnania. To znamená, že v je riadok, v ktorom zarovnanie prechádza cez stĺpec u matice. Optimálne zarovnanie môžeme nájsť spojením (konkatenáciou) optimálnych zarovnaní od $(0, 0)$ po (u, v) a od (u, v) po (m, n), kde m a n je pravá dolná bunka. Tým sme izolovali náš problém na dva samostatné problémy v ľavom hornom a pravom dolnom rohu matice. Potom môžeme rekurzívne rozdeľovať tieto podproblémy na menšie podproblémy. Na nájdenie v jednoducho sčítame prichádzajúce a odchádzajúce skóre pre daný stĺpec.



2. Lineárna pamäť (Hirschbergov algoritmus)

Hirschbergov algoritmus (z roku 1975) je geniálnym riešením jedného z najväčších problémov dynamického programovania: **ako nájsť optimálnu cestu bez toho, aby sme museli v pamäti držať celú obrovskú maticu.**

Motivácia: Prekážka menom $O(mn)$

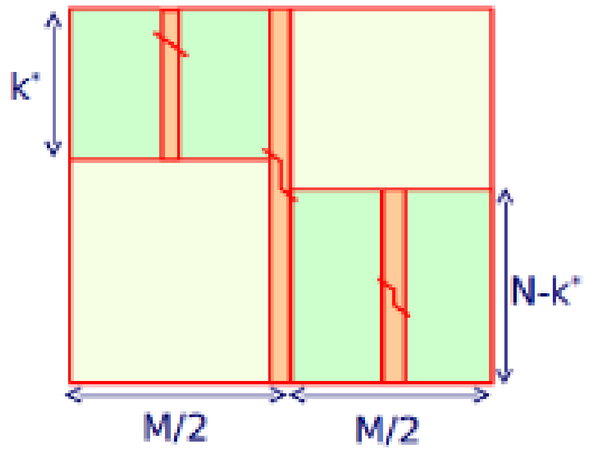
Pri klasickom Needleman-Wunschovom algoritme pre sekvencie dĺžky 10000 báz potrebujete maticu s 100000000 bunkami. Ak každá bunka (skóre + smerovník) zaberie 4 bajty, potrebujete **400 MB RAM**. To je v poriadku.

Ale pri zarovnávaní častí genómu s dĺžkou 100000 báz už potrebujete **40 GB RAM**, čo bežný počítač nezvládne. Hirschberg tento nárok znižuje na pár megabajtov (**lineárna pamäť**).

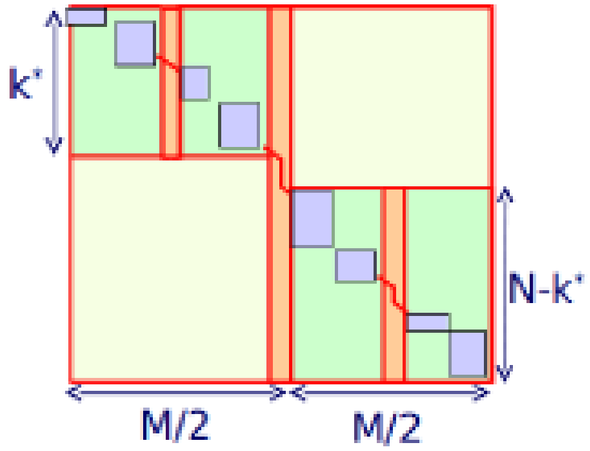
Kľúčové pozorovania

Hirschberg stojí na dvoch pilieroch:

- **Pozorovanie A (Skóre nepotrebuje minulosť):** Na výpočet hodnôt v stĺpci j potrebujete iba hodnoty zo stĺpca $j-1$. Akonáhle vypočítate stĺpec j, stĺpec $j-1$ môžete z pamäte vymazať. Tým získate **skóre** v lineárnom priestore $O(m)$.
- **Pozorovanie B (Obojsmernosť):** Ak rozdelíme maticu v strednom stĺpci $(n/2)$, optimálna cesta musí prejsť cez nejaký riadok i v tomto stĺpci. Ak poznáme tento bod $[i, n/2]$, môžeme problém rozdeliť na dva menšie.



Iterate procedure in corner quadrants



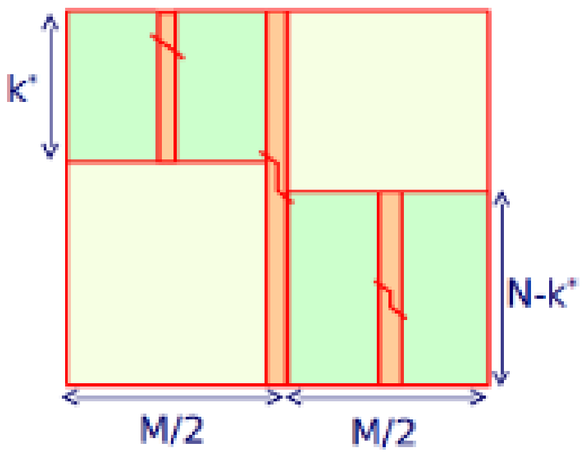
Total cost MN(1+½+¼+⅛+…)≤2MN

2. Úvod do rýchleho zarovnávania a vyhľadávania

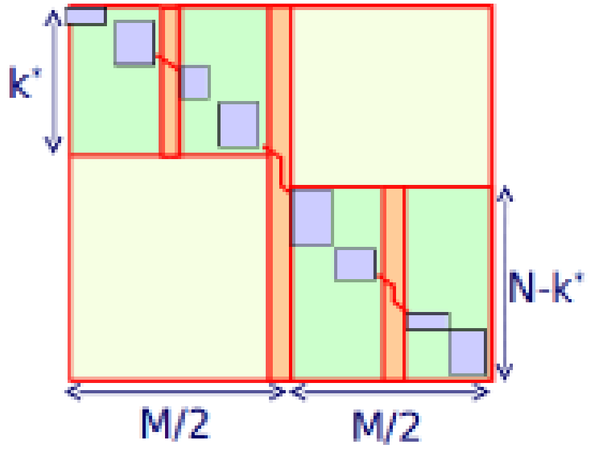
Mechanizmus: Rozdeľ a panuj (Divide and Conquer)

Algoritmus pracuje v týchto krokoch:

1. **Rozdelenie (Split):** Rozdelíme horizontálnu sekvenciu T presne na polovicu (stĺpec $W = n/2$).
2. **Dopredný a spätný beh:**
   - Spustíme DP zľava doprava (od začiatku po stredný stĺpec) a uložíme si len posledný stĺpec (vektor skóre L).
   - Spustíme DP sprava doľava (od konca po stredný stĺpec) a uložíme si len posledný stĺpec (vektor skóre R).
3. **Hľadanie priesečníka:** Sčítame tieto dva vektory: $Total_Score(i) = L(i) + R(i)$. Riadok i, kde je tento súčet maximálny, je bodom, v ktorom optimálna cesta pretína stred matice.
4. **Rekurzia:** Teraz máme dva menšie podproblémy:
   - Zarovnať ľavú hornú časť matice (pred priesečníkom).
   - Zarovnať pravú dolnú časť matice (za priesečníkom).
5. **Ukončenie:** Rekurzia končí, keď sú sekvencie také krátke, že ich vieme zarovnať triviálne.



Iterate procedure in corner quadrants



Total cost MN(1+½+¼+⅛+…)≤2MN

Analýza zložitosti (Dôkaz efektivity)

Priestorová zložitosť: $O(m)$

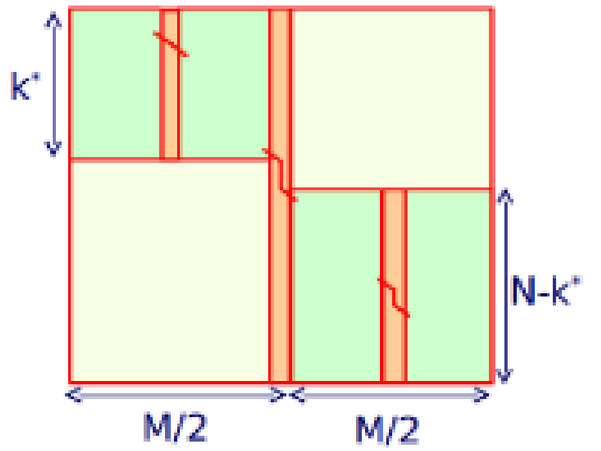
Ukladáme len vektory dĺžky m. Nepotrebujeme žiadne smerovníky (traceback pointers) pre celú maticu. Toto je hlavný cieľ algoritmu.

Časová zložitosť: $O(mn)$

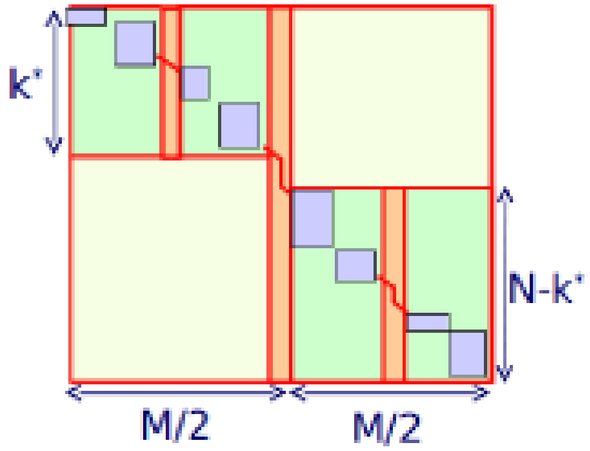
Na prvý pohľad sa zdá, že rekurzívne opakovanie výpočtov bude pomalé. Ale matematicky:

- V prvom kroku spracujeme celú plochu matice: $Area = m \times n$.
- V druhom kroku spracujeme dve podmatice, ktorých celková plocha je presne polovica pôvodnej: $Area / 2$.
- V treťom kroku: $Area / 4$.
- Súčet: $1 + 1/2 + 1/4 + 1/8 + \ldots \to 2$.

Výsledok: Hirschberg trvá presne **dvakrát dlhšie** než Needleman-Wunsch ($2 \times O(mn)$), čo je v asymptotickej notácii stále rovnaká zložitosť.



Iterate procedure in corner quadrants

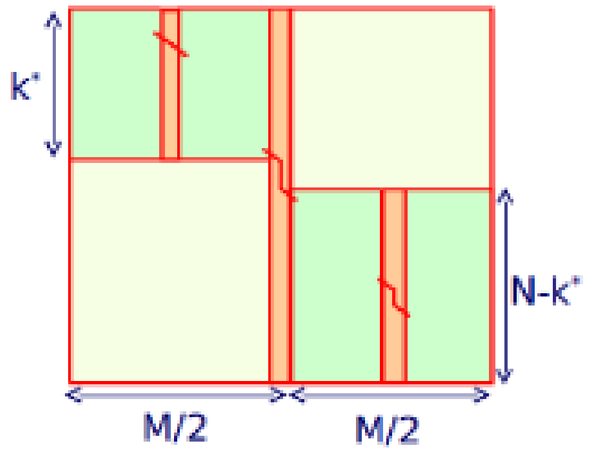


Total cost MN(1+½+¼+⅛+…)≤2MN

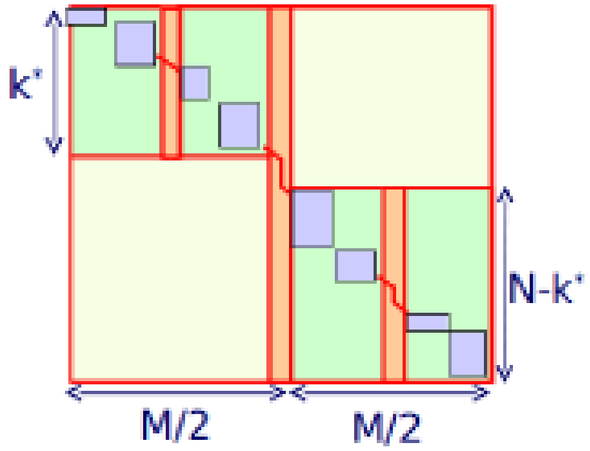
Lineárna pamäť (Hirschberg)

Hirschbergov algoritmus – Víťazstvo nad pamäťou

- **Hlavná myšlienka:** Spojenie dynamického programovania a stratégie „Rozdeľ a panuj“.
- **Vstup:** Dve sekvencie S (riadky) a T (stĺpce).
- **Proces:**
  1. Nájdenie optimálneho deliaceho bodu v strednom stĺpci matice.
  2. Využitie súčtu dopredného a spätného skóre (Prefix + Suffix).
  3. Rekurzívne delenie problému na menšie sub-matice.
- **Výhody:**
  - **Pamäť:** $O(m)$ namiesto $O(mn)$ – umožňuje zarovnávanie celých chromozómov.
  - **Presnosť:** Je to exaktný algoritmus (vždy nájde optimálne riešenie, na rozdiel od heuristík).
- **Cena za úsporu:** Výpočet trvá približne 2-krát dlhšie (konštantný faktor), ale zložitosť zostáva kvadratická.



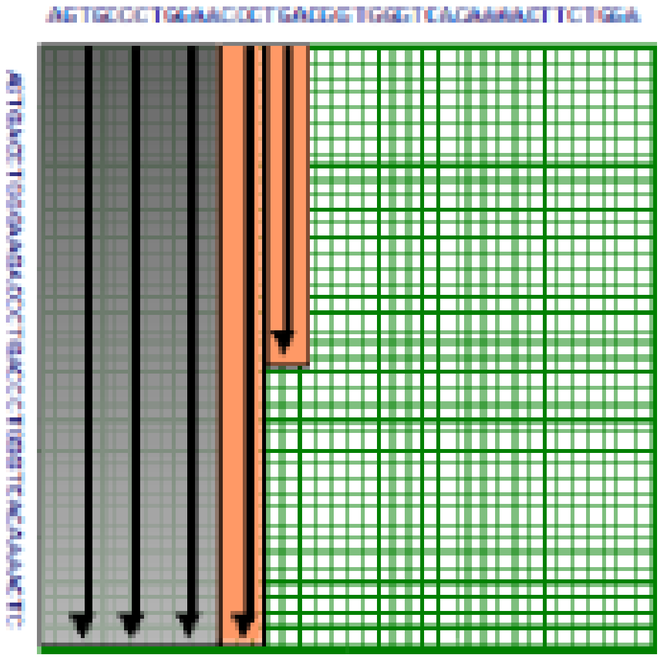
Iterate procedure in corner quadrants



Total cost MN(1+½+¼+⅛+…)≤2MN

2. Úvod do rýchleho zarovnávania a vyhľadávania

- Videli sme už skôr, že na výpočet optimálneho riešenia sme potrebovali uložiť skóre zarovnania v každej bunke, ako aj smerovník odrážajúci optimálnu voľbu vedúcu ku každej bunke.
- Ak nás však **zaujíma iba optimálne skóre zarovnania a nie samotné zarovnanie**, existuje metóda na výpočet riešenia pri súčasnej úspore priestoru.
- Na výpočet skóre ktorejkoľvek bunky potrebujeme iba skóre bunky nad ňou, vľavo a vľavo-diagonálne od aktuálnej bunky. Uložením predchádzajúceho a aktuálneho stĺpca, v ktorých počítame skóre, možno optimálne riešenie **vypočítať v lineárnom priestore.**



Generalizované penalizácie za medzery

Penalizácie za medzery určujú skóre vypočítané pre podsekvenciu, a tým ovplyvňujú, ktoré zarovnanie bude vybrané. Existuje niekoľko modelov, ktoré možno použiť, pričom každý z nich prináša iné algoritmické kompromisy.

- **Lineárna penalizácia za medzeru:** $w(k) = k \cdot p$

Bežným modelom je použitie lineárnej penalizácie za medzeru, kde každá jednotlivá medzera v sekvencii medzier dĺžky k je penalizovaná rovnako hodnotou p.

- **Kvadratická penalizácia za medzeru:** $w(k) = p + q \cdot k + r \cdot k^2$

V závislosti od situácie môže byť dobrým nápadom penalizovať odlišne napríklad medzery rôznych dĺžok. Pri kvadratickej penalizácii sa prírastková penalizácia kvadraticky znižuje s narastajúcou veľkosťou medzery. Daňou za to sú však náklady spojené s použitím zložitejšej funkcie penalizácie; čas behu sa podstatne zvyšuje. Tieto náklady možno zmierniť použitím jednoduchších aproximácií funkcií penalizácie medzier.

- **Afinná penalizácia za medzeru:** $w(k) = p + q \cdot k$**, kde** $q < p$

Afinná penalizácia je skvelým kompromisom: máte fixnú penalizáciu za začatie medzery a lineárne náklady na pridávanie k medzere.

- **Penalizácia za dĺžku medzery (mod 3)**

Môžete zvážiť aj komplexnejšie funkcie, ktoré berú do úvahy vlastnosti sekvencií kódujúcich proteíny. V prípade zarovnávania oblastí kódujúcich proteíny môže byť medzera s dĺžkou deliteľnou tromi (mod 3) penalizovaná menej, pretože by nespôsobila posun čítacieho rámca (frame shift).

Základy skórovacích matíc a Dayhoffovej model

Evolučný kontext

- **Cieľ:** Priradiť kvantitatívnu hodnotu podobnosti medzi aminokyselinami na základe evolučných pravidiel.
- **Margaret Dayhoff (1978):** Priekopníčka, ktorá vytvorila model založený na reálne pozorovaných mutáciách.
- **PAM (Accepted Point Mutation):** Bodová mutácia, ktorá prešla prirodzeným výberom a ustálila sa v druhu.

- Keď sú dva proteíny zarovnané, aké skóre by im malo byť priradené?
- Pre zarovnanie nukleotidov existovali špecifické skóre pre zhody a nezhody; ako boli odvodené? **Margaret Dayhoff (1966, 1978) poskytla model pravidiel, podľa ktorých dochádza k evolučným zmenám v proteínoch.**
- Dayhoffovej model má 7 krokov (podľa článku Dayhoff, 1978).
- **Poskytuje základ kvantitatívneho skórovacieho systému pre párové zarovnania medzi akýmikoľvek proteínmi, či už sú blízko alebo vzdialene príbuzné.**

Dayhoffovej model, 1. krok (zo 7): Akceptované bodové mutácie

- Dayhoffová a kolegovia zvažovali problém, ako **priradiť skóre zarovnaným aminokyselinovým zvyškom**.
- **Ich prístupom bolo katalogizovať stovky proteínov a porovnať sekvencie blízko príbuzných proteínov v mnohých rodinách.** Zaoberali sa otázkou, ktoré konkrétne aminokyselinové substitúcie pozorujeme, keď sú zarovnané dve homológne proteínové sekvencie.
- Definovali **akceptovanú bodovú mutáciu** ako nahradenie jednej aminokyseliny v proteíne iným zvyškom, ktorý bol akceptovaný prirodzeným výberom. Akceptovaná bodová mutácia sa skracuje ako **PAM** (z angl. Accepted Point Mutation).
- K zmene aminokyseliny akceptovanej prirodzeným výberom dochádza, keď:
  - (1) gén podstúpi mutáciu DNA tak, že kóduje inú aminokyselinu; a
  - (2) celý druh prijme túto zmenu ako prevládajúcu formu proteínu.

Ktoré bodové mutácie sú v evolúcii proteínov akceptované?

Intuitívne by boli najľahšie akceptované konzervatívne náhrady, ako napríklad serín za treonín. Aby určili všetky možné zmeny, Dayhoffová a kolegovia preskúmali 1572 zmien v 71 skupinách blízko príbuzných proteínov. Ich **definícia „akceptovaných“ mutácií bola preto založená na empiricky pozorovaných substitúciách aminokyselín.** Ich prístup zahŕňal fylogenetickú analýzu: namiesto priameho porovnávania dvoch aminokyselinových zvyškov ich porovnávali s odvodeným spoločným predkom týchto sekvencií.

Empirické výsledky pozorovaných substitúcií opisujú frekvenciu, s akou sú akékoľvek páry aminokyselín zarovnané.

Dayhoffovej model, 1. krok (zo 7): Akceptované bodové mutácie

Dayhoffovej prístup k určovaniu aminokyselinových substitúcií.

(a) Čiastočné viacnásobné sekvenčné zarovnanie ľudského alfa 1 globínu, beta globínu, delta globínu a myoglobínu. Štyri stĺpce, v ktorých majú alfa 1 globín a myoglobín odlišné aminokyselinové zvyšky, sú vyznačené červenou farbou. Napríklad A je zarovnané s G (šípka).

(b) Fylogenetický strom, ktorý zobrazuje štyri existujúce sekvencie (označené 1–4), ako aj dva vnútorné uzly, ktoré predstavujú ancestrálne (predkovské) sekvencie (označené 5 a 6). Odvodené ancestrálne sekvencie boli identifikované pomocou analýzy maximálnej parsimónie (úspornosti) s použitím softvéru PAUP sú zobrazené v časti (a).

Z tejto analýzy je zrejmé, že v každom zo stĺpcov označených červenou farbou **nedošlo k priamej zámene dvoch aminokyselín medzi alfa 1 globínom a myoglobínom.** Namiesto toho došlo k divergencii ancestrálneho zvyšku. Napríklad šípka v (a) označuje ancestrálny glutamát, ktorý sa vyvinul tak, že sa stal alanínom alebo glycínom, ale nebolo by správne tvrdiť, že alanín bol priamo premenený na glycín.

Dayhoffovej model, 1. krok (zo 7): Akceptované bodové mutácie

- Tabuľky odhaľuje, **ktoré substitúcie sú nepravdepodobné** (napríklad cysteín a tryptofán majú znateľne málo substitúcií), zatiaľ čo iné, ako asparagín a serín, **tolerujú náhrady celkom bežne.**
- Počty akceptovaných bodových mutácií, vynásobené desiatimi, v 1572 prípadoch aminokyselinových substitúcií z blízko príbuzných proteínových sekvencií. **Aminokyseliny sú zoradené abecedne podľa ich trojpísmenových kódov.** Všimnite si, že niektoré substitúcie (zeleno podfarbené políčka) sú akceptované veľmi bežne (napríklad V a I alebo S a T). Iné aminokyseliny, ako C a W, sú len zriedkavo nahradzované akýmkoľvek iným zvyškom (oranžovo podfarbené políčka).

	A Ala	R Arg	N Asn	D Asp	C Cys	Q Gln	E Glu	G Gly	H His	I Ile	L Leu	K Lys	M Met	F Phe	P Pro	S Ser	T Thr	W Trp	Y Tyr	V Val
A																				
R	30																			
N	109	17																		
D	154	0	532																	
C	33	10	0	0																
Q	93	120	50	76	0															
E	266	0	94	831	0	422														
G	579	10	156	162	10	30	112													
H	21	103	226	43	10	243	23	10												
I	66	30	36	13	17	8	35	0	3											
L	95	17	37	0	y	75	15	17	40	253										
K	57	477	322	85	0	147	104	60	23	43	39									
M	29	17	0	0	0	20	7	7	0	57	207	90								
F	20	7	7	0	0	0	0	17	20	90	167	0	17							
P	345	67	27	10	10	93	40	49	50	7	43	43	4	7						
S	772	137	432	98	117	47	86	450	26	20	32	168	20	40	269					
T	590	20	169	57	10	37	31	50	14	129	52	200	28	10	73	696				
W	0	27	3	0	0	0	0	0	3	0	13	0	0	10	0	17	0			
Y	20	3	36	0	30	0	10	0	40	13	23	10	0	260	0	22	23	6		
V	365	20	13	17	33	27	37	97	30	661	303	17	77	10	50	43	186	0	17	
	A Ala	R Arg	N Asn	D Asp	C Cys	Q Gln	E Glu	G Gly	H His	I Ile	L Leu	K Lys	M Met	F Phe	P Pro	S Ser	T Thr	W Trp	Y Tyr	V Val

Dayhoffovej model, 2. krok (zo 7): Frekvencia aminokyselín

Na modelovanie pravdepodobnosti, že sa jedna zarovnaná aminokyselina v proteíne zmení na inú, potrebujeme poznať frekvencie výskytu každej aminokyseliny.

Normalizované frekvencie aminokyselín.

- Súčet týchto hodnôt je 1.
- Ak by bolo všetkých 20 aminokyselín v proteínoch zastúpených rovnako, tieto hodnoty by boli všetky 0,05 (t. j. 5 %); namiesto toho sa aminokyseliny vo frekvencii svojho výskytu líšia.

Gly	0.089	Arg	0.041
Ala	0.087	Asn	0.040
Leu	0.085	Phe	0.040
Lys	0.081	Gln	0.038
Ser	0.070	Ile	0.037
Val	0.065	His	0.034
Thr	0.058	Cys	0.033
Pro	0.051	Tyr	0.030
Glu	0.050	Met	0.015
Asp	0.047	Trp	0.010

Dayhoffovej model, 3. krok (zo 7): Relatívna mutabilita aminokyselín

- **Dayhoffová a kol. vypočítali relatívne mutability aminokyselín.**
- Mutabilita popisuje, ako často sa každá aminokyselina pravdepodobne zmení počas krátkeho evolučného obdobia. Na výpočet relatívnej mutability vydelili počet pozorovaných mutácií každej aminokyseliny (m_i) celkovou frekvenciou výskytu tejto aminokyseliny (f_i).

- **Prečo sú niektoré aminokyseliny mutabilnejšie ako iné?** Menej mutabilné zvyšky majú pravdepodobne dôležité štrukturálne alebo funkčné úlohy v proteínoch, takže následok ich nahradenia akýmkoľvek iným zvyškom by mohol byť pre organizmus škodlivý. Naopak, najviac mutabilné aminokyseliny – asparagín, serín, kyselina asparágová a kyselina glutámová – majú v proteínoch funkcie, ktoré ľahko preberajú iné zvyšky.
- Substitúcie, ktoré sa vyskytujú v proteínoch, možno pochopiť aj s odkazom na genetický kód. Všimnite si, ako bežné substitúcie aminokyselín majú tendenciu vyžadovať iba jednu nukleotidovú zmenu. **Nízka mutabilita určitých aminokyselín (ako tryptofán) naznačuje, že substitúcie nie sú tolerované prirodzeným výberom.**

Asn	134	His	66
Ser	120	Arg	65
Asp	106	Lys	56
Glu	102	Pro	56
Ala	100	Gly	49
Thr	97	Tyr	41
Ile	96	Phe	41
Met	94	Leu	40
Gln	93	Cys	20
Val	74	Trp	18

Dayhoffovej model, 4. krok (zo 7): Matica pravdepodobnosti mutácie pre evolučnú vzdialenosť 1 PAM

- Dayhoffová a kolegovia ďalej použili údaje o akceptovaných mutáciách a pravdepodobnosti výskytu každej aminokyseliny na vygenerovanie matice pravdepodobnosti mutácie . Každý prvok maticec M_{ij} ukazuje pravdepodobnosť, že pôvodná aminokyselina (stĺpce) bude nahradená inou aminokyselinou (riadky) počas definovaného evolučného intervalu.
- **V tomto prípade je interval jeden PAM, ktorý je definovaný ako jednotka evolučnej divergencie,** v ktorej sa zmenilo 1 % aminokyselín medzi dvoma proteínovými sekvenciami.

		Original amino acid																			
		A Ala	R Arg	N Asn	D Asp	C Cys	Q Gln	E Glu	G Gly	H His	I Ile	L Leu	K Lys	M Met	F Phe	P Pro	S Ser	T Thr	W Trp	Y Tyr	V Val
Replacement amino acid	A	98.7	0.0	0.1	0.1	0.0	0.1	0.2	0.2	0.0	0.1	0.0	0.0	0.1	0.0	0.2	0.4	0.3	0.0	0.0	0.2
	R	0.0	99.1	0.0	0.0	0.0	0.1	0.0	0.0	0.1	0.0	0.0	0.2	0.0	0.0	0.0	0.1	0.0	0.1	0.0	0.0
	N	0.0	0.0	98.2	0.4	0.0	0.0	0.1	0.1	0.2	0.0	0.0	0.1	0.0	0.0	0.0	0.2	0.1	0.0	0.0	0.0
	D	0.1	0.0	0.4	98.6	0.0	0.1	0.5	0.1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.1	0.0	0.0	0.0	0.0
	C	0.0	0.0	0.0	0.0	99.7	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.1	0.0	0.0	0.0	0.0
	Q	0.0	0.1	0.0	0.1	0.0	98.8	0.3	0.0	0.2	0.0	0.0	0.1	0.0	0.0	0.1	0.0	0.0	0.0	0.0	0.0
	E	0.1	0.0	0.1	0.6	0.0	0.4	98.7	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
	G	0.2	0.0	0.1	0.1	0.0	0.0	0.1	99.4	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.2	0.0	0.0	0.0	0.1
	H	0.0	0.1	0.2	0.0	0.0	0.2	0.0	0.0	99.1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
	I	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	98.7	0.1	0.0	0.2	0.1	0.0	0.0	0.1	0.0	0.0	0.3
	L	0.0	0.0	0.0	0.0	0.0	0.1	0.0	0.0	0.0	0.2	99.5	0.0	0.5	0.1	0.0	0.0	0.0	0.0	0.0	0.2
	K	0.0	0.4	0.3	0.1	0.0	0.1	0.1	0.0	0.0	0.0	0.0	99.3	0.2	0.0	0.0	0.1	0.1	0.0	0.0	0.0
	M	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.1	0.1	0.0	98.7	0.0	0.0	0.0	0.0	0.0	0.0	0.0
	F	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.1	0.1	0.0	0.0	99.5	0.0	0.0	0.0	0.0	0.3	0.0
	P	0.1	0.1	0.0	0.0	0.0	0.1	0.0	0.0	0.1	0.0	0.0	0.0	0.0	0.0	99.3	0.1	0.0	0.0	0.0	0.0
	S	0.3	0.1	0.3	0.1	0.1	0.0	0.1	0.2	0.0	0.0	0.0	0.1	0.0	0.0	0.2	98.4	0.4	0.1	0.0	0.0
	T	0.2	0.0	0.1	0.0	0.0	0.0	0.0	0.0	0.0	0.1	0.0	0.1	0.1	0.0	0.1	0.3	98.7	0.0	0.0	0.1
	W	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	99.8	0.0	0.0
	Y	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.2	0.0	0.0	0.0	0.0	99.5	0.0
	V	0.1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.6	0.1	0.0	0.2	0.0	0.0	0.0	0.1	0.0	0.0	99.0

Dayhoffovej model, 4. krok (zo 7): Matica pravdepodobnosti mutácie pre evolučnú vzdialenosť 1 PAM

- Najvyššie skóre sú distribuované na diagonále. Hodnota 98,7 vľavo hore naznačuje, že keď pôvodná sekvencia pozostáva z alanínu, existuje 98,7 % pravdepodobnosť, že náhradná aminokyselina bude tiež alanín počas evolučnej vzdialenosti jedného PAM.

Prvky matice M_{ij} označujú pravdepodobnosť, že pôvodná aminokyselina j bude nahradená aminokyselinou z riadku i. A_{ij} je prvok matice akceptovaných bodových mutácií – prvý krok, napríklad hodnota zodpovedajúca nahradeniu pôvodného alanínu arginínom.

Prvky mimo diagonály sa vypočítajú ako: $M_{ij} = \dfrac{\lambda\, m_j\, A_{ij}}{\sum_i A_{ij}}$

kde je λ konštanta proporcionality a m_j je mutabilita j-tej aminokyseliny. Diagonálne prvky sú: $M_{ij} = 1 - \lambda m_j$, kde M_{ij} označujú pravdepodobnosť, že pôvodná aminokyselina j zostane j bez toho, aby došlo k jej nahradeniu inou aminokyselinou.

Porozumieme týmto dvom rovniciam pomocou preskúmania prvého stĺpca matice pravdepodobnosti mutácie, v ktorej je pôvodnou aminokyselinou alanín. Celková pravdepodobnosť (súčet všetkých prvkov) je 1 alebo, ak uvažujeme o prvkoch ako o percentách, súčet stĺpca je 100 %. Intuitívne je rozumné, že pravdepodobnosť pozorovania zmeny aminokyseliny – ekvivalentná súčtu všetkých prvkov okrem alanínu, ktorý zostáva sám sebou, M_{ij} – je úmerná mutabilite alanínu.

Tieto údaje tvoria základ skórovacieho systému, v ktorom sú rozumné substitúcie odmeňované, zatiaľ čo nepravdepodobné penalizované.

Replaced amino acid	A Ala	R Arg	N Asn	D Asp	C Cys	Q Gln	E Glu	G Gly	H His
A	98.7	0.0	0.1	0.1	0.0	0.1	0.2	0.2	0.0
R	0.0	99.1	0.0	0.0	0.0	0.1	0.0	0.0	0.1
N	0.0	0.0	98.2	0.4	0.0	0.0	0.1	0.1	0.2
D	0.1	0.0	0.4	98.6	0.0	0.1	0.5	0.1	0.0
C	0.0	0.0	0.0	0.0	99.7	0.0	0.0	0.0	0.0
Q	0.0	0.1	0.0	0.1	0.0	98.8	0.3	0.0	0.2
E	0.1	0.0	0.1	0.6	0.0	0.4	98.7	0.0	0.0
G	0.2	0.0	0.1	0.1	0.0	0.0	0.1	99.4	0.0
H	0.0	0.1	0.2	0.0	0.0	0.2	0.0	0.0	99.1
I	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
L	0.0	0.0	0.0	0.0	0.0	0.1	0.0	0.0	0.0
K	0.0	0.4	0.3	0.1	0.0	0.1	0.1	0.0	0.0
M	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
F	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
P	0.1	0.1	0.0	0.0	0.0	0.1	0.0	0.0	0.1
S	0.3	0.1	0.3	0.1	0.1	0.0	0.1	0.2	0.0
T	0.2	0.0	0.1	0.0	0.0	0.0	0.0	0.0	0.0
W	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Y	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
V	0.1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

Dayhoffovej model, 5. krok (zo 7): PAM250 a ďalšie matice PAM

Matica PAM1 bola založená na blízko príbuzných sekvenciách (1 % zmena). Často nás zaujímajú vzťahy proteínov, ktoré zdieľajú oveľa menej identity. Matice pre väčšie vzdialenosti sa neodvodzujú len úpravou, ale maticovým násobením: matica PAM1 sa vynásobí sama sebou (napr. 250-krát pre PAM250), čím sa extrapoluje z PAM1.

Matica PAM250 zodpovedá vzdialenosti, kde proteíny zdieľajú približne 20 % identity. V porovnaní s PAM1 sa v nej stráca veľa informačného obsahu. Napríklad pre alanín je pri PAM250 len 13 % šanca, že zostane alanínom, a takmer rovnaká šanca (12 %), že bude nahradený glycínom.

		Original amino acid																			
		A	R	N	D	C	Q	E	G	H	I	L	K	M	F	P	S	T	W	Y	V
Replacement amino acid	A	13	6	9	9	5	8	9	12	6	8	6	7	7	4	11	11	11	2	4	9
	R	3	17	4	3	2	5	3	2	6	3	2	9	4	1	4	4	3	7	2	2
	N	4	4	6	7	2	5	6	4	6	3	2	5	3	2	4	5	4	2	3	3
	D	5	4	8	11	1	7	10	5	6	3	2	5	3	1	4	5	5	1	2	3
	C	2	1	1	1	52	1	1	2	2	2	1	1	1	1	2	3	2	1	4	2
	Q	3	5	5	6	1	10	7	3	7	2	3	5	3	1	4	3	3	1	2	3
	E	5	4	7	11	1	9	12	5	6	3	2	5	3	1	4	5	5	1	2	3
	G	12	5	10	10	4	7	9	27	5	5	4	6	5	3	8	11	9	2	3	7
	H	2	5	5	4	2	7	4	2	15	2	2	3	2	2	3	3	2	2	3	2
	I	3	2	2	2	2	2	2	2	2	10	6	2	6	5	2	3	4	1	3	9
	L	6	4	4	3	2	6	4	3	5	15	34	4	20	13	5	4	6	6	7	13
	K	6	18	10	8	2	10	8	5	8	5	4	24	9	2	6	8	8	4	3	5
	M	1	1	1	1	0	1	1	1	1	2	3	2	6	2	1	1	1	1	1	2
	F	2	1	2	1	1	1	1	1	3	5	6	1	4	32	1	2	2	4	20	3
	P	7	5	5	4	3	5	4	5	5	3	3	4	3	2	20	6	5	1	2	4
	S	9	6	8	7	7	6	7	9	6	5	4	7	5	3	9	10	9	4	4	6
	T	8	5	6	6	4	5	5	6	4	6	4	6	5	3	6	8	11	2	3	6
	W	0	2	0	0	0	0	0	0	1	0	1	0	0	1	0	1	0	55	1	0
	Y	1	1	2	1	3	1	1	1	3	2	2	1	2	15	1	2	2	3	31	2
	V	7	4	4	4	4	4	4	5	4	15	10	4	10	5	5	5	7	2	4	17

Dayhoffovej model, 6. krok (zo 7): Od matice pravdepodobnosti mutácie k matici šancí príbuznosti

Dayhoffová a kol. definovali maticu šancí príbuznosti (relatedness odds matrix) ako pomer: $R_{ij} =\frac {M_ij}/{f_i}

Hodnota znamená, že substitúcia nastáva tak často, ako sa dá očakávať náhodou. Hodnoty naznačujú, že zarovnanie nastáva častejšie než náhodou (konzervatívna zmena).

Dayhoffovej model, 7. krok (zo 7): Log-Odds skórovacia mati

Logaritmická forma matice šancí sa nazýva log-odds matica: $s_{ij} = 10 \times \log_{10}\left(\frac{M_{ij}}{f_i}\right).$

Použitie logaritmu je výhodné, pretože umožňuje sčítať skóre zarovnaných zvyškov (namiesto násobenia pravdepodobností).

original amino acid

replacement amino acid

PAM0	A	R	N	D	C	Q	E	G
A	100	0	0	0	0	0	0	0
R	0	100	0	0	0	0	0	0
N	0	0	100	0	0	0	0	0
D	0	0	0	100	0	0	0	0
C	0	0	0	0	100	0	0	0
Q	0	0	0	0	0	100	0	0
E	0	0	0	0	0	0	100	0
G	0	0	0	0	0	0	0	100

original amino acid

PAM∞	A	R	N	D	C	Q	E	G
A	8.7	8.7	8.7	8.7	8.7	8.7	8.7	8.7
R	4.1	4.1	4.1	4.1	4.1	4.1	4.1	4.1
N	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0
D	4.7	4.7	4.7	4.7	4.7	4.7	4.7	4.7
C	3.3	3.3	3.3	3.3	3.3	3.3	3.3	3.3
Q	3.8	3.8	3.8	3.8	3.8	3.8	3.8	3.8
E	5.0	5.0	5.0	5.0	5.0	5.0	5.0	5.0
G	8.9	8.9	8.9	8.9	8.9	8.9	8.9	8.9

A	2																		
R	-2	6																	
N	0	0	2																
D	0	-1	2	4															
C	-2	-4	-4	-5	12														
Q	0	1	1	2	-5	4													
E	0	-1	1	3	-5	2	4												
G	1	-3	0	1	-3	-1	0	5											
H	-1	2	2	1	-3	3	1	-2	6										
I	-1	-2	-2	-2	-2	-2	-2	-3	-2	5									
L	-2	-3	-3	-4	-6	-2	-3	-4	-2	-2	6								
K	-1	3	1	0	-5	1	0	-2	0	-2	-3	5							
M	-1	0	-2	-3	-5	-1	-2	-3	-2	2	4	0	6						
F	-3	-4	-3	-6	-4	-5	-5	-5	-2	1	2	-5	0	9					
P	1	0	0	-1	-3	0	-1	0	0	-2	-3	-1	-2	-5	6				
S	1	0	1	0	0	-1	0	1	-1	-1	-3	0	-2	-3	1	2			
T	1	-1	0	0	-2	-1	0	0	-1	0	-2	0	-1	-3	0	1	3		
W	-6	2	-4	-7	-8	-5	-7	-7	-3	-5	-2	-3	-4	0	-6	-2	-5	17	
Y	-3	-4	-2	-4	0	-4	-4	-5	0	-1	-1	-4	-2	7	-5	-3	-3	0	10
V	0	-2	-2	-2	-2	-2	-2	-1	-2	4	2	-2	2	-1	-1	-1	0	-6	-2	4
	A	**R**	**N**	**D**	**C**	**Q**	**E**	**G**	**H**	**I**	**L**	**K**	**M**	**F**	**P**	**S**	**T**	**W**	**Y**	**V**

Dayhoffovej model, 6. krok (zo 7): Od matice pravdepodobnosti mutácie k matici šancí príbuznosti

Dayhoffová a kol. definovali maticu šancí príbuznosti (relatedness odds matrix) ako pomer:

$$R_{ij} = \frac{M_{ij}}{f_i \quad i}$$

Hodnota znamená, že substitúcia nastáva tak často, ako sa dá očakávať náhodou. Hodnoty naznačujú, že zarovnanie nastáva častejšie než náhodou (konzervatívna zmena).

Dayhoffovej model, 7. krok (zo 7): Log-Odds skórovacia matica

Logaritmická forma matice šancí sa nazýva log-odds matica:

$$s_{ij} = 10 \times \log_{10}\left(\frac{M_{ij}}{f_i}\right).$$

Použitie logaritmu je výhodné, pretože umožňuje sčítať skóre zarovnaných zvyškov (namiesto násobenia pravdepodobností).

Táto matica je symetrická a hodnoty v nej sa zvyčajne zaokrúhľujú na celé čísla.

A	7																			
R	-10	9																		
N	-7	-9	9																	
D	-6	-17	-1	8																
C	-10	-11	-17	-21	10															
Q	-7	-4	-7	-6	-20	9														
E	-5	-15	-5	0	-20	-1	8													
G	-4	-13	-6	-6	-13	-10	-7	7												
H	-11	-4	-2	-7	-10	-2	-9	-13	10											
I	-8	-8	-8	-11	-9	-11	-8	-17	-13	9										
L	-9	-12	-10	-19	-21	-8	-13	-14	-9	-4	7									
K	-10	-2	-4	-8	-20	-6	-7	-10	-10	-9	-11	7								
M	-8	-7	-15	-17	-20	-7	-10	-12	-17	-3	-2	-4	12							
F	-12	-12	-12	-21	-19	-19	-20	-12	-9	-5	-5	-20	-7	9						
P	-4	-7	-9	-12	-11	-6	-9	-10	-7	-12	-10	-10	-11	-13	8					
S	-3	-6	-2	-7	-6	-8	-7	-4	-9	-10	-12	-7	-8	-9	-4	7				
T	-3	-10	-5	-8	-11	-9	-9	-10	-11	-5	-10	-6	-7	-12	-7	-2	8			
W	-2	-5	-11	-21	-22	-19	-23	-21	-10	-20	-9	-18	-19	-7	-20	-8	-19	13		
Y	-11	-14	-7	-17	-7	-18	-11	-20	-6	-9	-10	-12	-17	-1	-20	-10	-9	-8	10	
V	-5	-11	-12	-11	-9	-10	-10	-9	-9	-1	-5	-13	-4	-12	-9	-10	-6	-22	-10	8
	A	R	N	D	C	Q	E	G	H	I	L	K	M	F	P	S	T	W	Y	V

FIGURE 3.15 Log-odds matrix for PAM10. Low PAM values such as this are useful for aligning very closely related sequences. Compare this with the PAM250 matrix (**Fig. 3.14**) and note that there are larger positive scores for identical matches in this PAM10 matrix and larger penalties for mismatches. Adapted from NCBI, ftp://ftp.ncbi.nlm.nih.gov/blast/matrices/.

A	4																			
R	-1	5																		
N	-2	0	6																	
D	-2	-2	1	6																
C	0	-3	-3	-3	9															
Q	-1	1	0	0	-3	5														
E	-1	0	0	2	-4	2	5													
G	0	-2	0	-1	-3	-2	-2	6												
H	-2	0	1	-1	-3	0	0	-2	8											
I	-1	-3	-3	-3	-1	-3	-3	-4	-3	4										
L	-1	-2	-3	-4	-1	-2	-3	-4	-3	2	4									
K	-1	2	0	-1	-1	1	1	-2	-1	-3	-2	5								
M	-1	-2	-2	-3	-1	0	-2	-3	-2	1	2	-1	5							
F	-2	-3	-3	-3	-2	-3	-3	-3	-1	0	0	-3	0	6						
P	-1	-2	-2	-1	-3	-1	-1	-2	-2	-3	-3	-1	-2	-4	7					
S	1	-1	1	0	-1	0	0	0	-1	-2	-2	0	-1	-2	-1	4				
T	0	-1	0	-1	-1	-1	-1	-2	-2	-1	-1	-1	-1	-2	-1	1	5			
W	-3	-3	-4	-4	-2	-2	-3	-2	-2	-3	-2	-3	-1	1	-4	-3	-2	11		
Y	-2	-2	-2	-3	-2	-1	-2	-3	2	-1	-1	-2	-1	3	-3	-2	-2	2	7	
V	0	-3	-3	-3	-1	-2	-2	-3	-3	3	1	-2	1	-1	-2	-2	0	-3	-1	4
	A	**R**	**N**	**D**	**C**	**Q**	**E**	**G**	**H**	**I**	**L**	**K**	**M**	**F**	**P**	**S**	**T**	**W**	**Y**	**V**

FIGURE 3.17 The BLOSUM62 scoring matrix of Henikoff and Henikoff (1992). This matrix merges all proteins in an alignment that have 62% amino acid identity or greater into one sequence. BLOSUM62 performs better than alternative BLOSUM matrices or a variety of PAM matrices at detecting distant relationships between proteins. It is therefore the default scoring matrix for most database search programs such as BLAST (Chapter 4).

Source: Henikoff & Henikoff (1992). Reproduced with permission from S. Henikoff.